PROGRAM DOCUMENTATION

1. Program Title:

Subarray with Given Sum (Optimized using Prefix Sum and HashMap)

2. Description

This program finds the total number of continuous subarrays whose sum is equal to a given target value. It uses the Prefix Sum technique along with a HashMap to efficiently calculate the result.

3. Purpose of the Program

The purpose of this program is to count the number of continuous subarrays whose sum equals the given target in O(n) time complexity, even when the array contains negative numbers.

4. Input

First input: Integer `n` (size of array)

Second input: `n` long integers (array elements)

Third input: Long integer `target` (required subarray sum)

5. Output

Prints the total number of continuous subarrays whose sum equals `target`.

6. Algorithm / Logic

1. Create a `HashMap` to store prefix sum frequencies.
2. Insert `(0 to 1)` into the map to handle subarrays starting from index 0.
3. Initialize `prefixSum = 0` and `count = 0`.
4. Traverse the array:

    Add current element to `prefixSum`.

    Check if `(prefixSum - target)` exists in the map.

    If it exists, add its frequency to `count`.

    Update the frequency of `prefixSum` in the map.

5. Return `count`.

7. Variables Used

`arr[]`	=	Stores array elements
`target`	=	Required subarray sum
`prefixSum`	=	Stores cumulative sum
`count`	=	Stores number of valid subarrays
`map`	=	Stores prefix sum frequencies
`n`	=	Size of array

8. Functions / Methods Used

- `countSubarrays()` = Calculates total valid subarrays
- `main()` = Handles input and output
- `HashMap.put()` = Inserts key-value pair
- `HashMap.get()`= Retrieves value
- `HashMap.containsKey()` = Checks if key exists
- `HashMap.getOrDefault()` = Gets value or default
- `Scanner` = Takes user input

9. Sample Input

5
1 2 3 -2 2
3

10. Sample Output

4

11. Time Complexity

O(n) = Each element is processed once.

12. Space Complexity

O(n) = HashMap may store up to `n` prefix sums.

13. Java Program (With Comments)

```
import java.util.*;
public class Subarray {
    // Method to count subarrays with given target sum
    public static int countSubarrays(long[] arr, long target) {
        // HashMap to store prefix sum frequencies
        HashMap<Long, Integer> map = new HashMap<>();
        long prefixSum = 0;   // Stores cumulative sum
        int count = 0;        // Stores result (number of subarrays)
        // Initialize map with prefixSum 0 having frequency 1
        // This handles subarrays starting from index 0
        map.put(0L, 1);
        // Traverse the array
        for(int i = 0; i < arr.length; i++) {
            // Add current element to prefixSum
            prefixSum += arr[i];
            // Check if (prefixSum - target) exists in map
            // If yes, add its frequency to count
            if(map.containsKey(prefixSum - target)) {
                count += map.get(prefixSum - target);
            }
            // Update prefixSum frequency in map
            map.put(prefixSum, map.getOrDefault(prefixSum, 0) + 1);
        }
        // Return total subarrays count
        return count;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();  // Read size of array
        long[] arr = new long[n];
        // Read array elements
        for(int i = 0; i < n; i++) {
            arr[i] = sc.nextLong();
        }
        long target = sc.nextLong();  // Read target sum
        // Call method to compute result
        int result = countSubarrays(arr, target);

        // Print result
        System.out.println(result);
    }
}
```

---

1. Program Title

Minimum Meeting Rooms Using Two Pointer Technique

2. Description

This program determines the minimum number of meeting rooms required so that no meetings overlap. It separates start and end times, sorts them, and uses a two-pointer technique to calculate the maximum number of overlapping meetings at any time.

3. Purpose of the Program

The purpose of this program is to efficiently calculate the minimum number of meeting rooms required to conduct all meetings without any time conflict.

4. Input

First input: Integer `n` (number of meetings)

Next n lines: Two integers representing `start` and `end` time of each meeting

5. Output (O/P)

Prints the minimum number of meeting rooms required.

6. Algorithm / Logic

1. Read number of meetings `n`.
2. Store all meeting intervals in a 2D array.
3. Create two separate arrays:

   `start[]` = stores start times

   `end[]` = stores end times

4. Sort both arrays.
5. Use two pointers `i` and `j`:
   - If `start[i] < end[j]`, increment rooms (new meeting starts).
   - Else, decrement rooms (meeting ended).
6. Keep track of maximum rooms required.
7. Return `maxRooms`.

---

7. Variables Used

Variable	Purpose

`intervals[][]`	Stores meeting start and end times
`start[]`	Stores all start times
`end[]`	Stores all end times
`rooms`	Current number of rooms in use
`maxRooms`	Maximum rooms required
`i`	Pointer for start array
`j`	Pointer for end array
`n`	Number of meetings
`sc`	Used for input

8. Functions / Methods Used

`minMeetingRooms()` = Calculates minimum rooms required

`main()` = Handles input and output

`Arrays.sort()` = Sorts arrays

`Math.max()` = Finds maximum value

`Scanner` = Takes user input

9. Sample Input

3
0 30
5 10
15 20

10. Sample Output

2

11. Time Complexity

O(n log n)
Sorting of start and end arrays takes `O(n log n)` time.

12. Space Complexity

O(n)
Extra space used for start and end arrays

13. Java Program (With Comments)

```
import java.util.*;

public class Meeting {

    // Method to calculate minimum meeting rooms required
    public static int minMeetingRooms(int[][] intervals) {

        int n = intervals.length;

        // Arrays to store start and end times separately
        int[] start = new int[n];
        int[] end = new int[n];

        // Extract start and end times
        for(int i = 0; i < n; i++) {
            start[i] = intervals[i][0];
            end[i] = intervals[i][1];
        }

        // Sort both arrays
        Arrays.sort(start);
        Arrays.sort(end);

        int rooms = 0;      // Current rooms in use
        int maxRooms = 0;   // Maximum rooms required
        int i = 0, j = 0;   // Two pointers

        // Traverse through meetings
        while(i < n) {

            // If meeting starts before previous ends
            if(start[i] < end[j]) {
                rooms++;  // Allocate new room
                maxRooms = Math.max(maxRooms, rooms);
                i++;
            }
            else {
                rooms--;  // Free one room
                j++;
            }
        }

        return maxRooms;  // Return minimum rooms required
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();  // Number of meetings
        int[][] intervals = new int[n][2];

        // Read meeting intervals
        for(int i = 0; i < n; i++) {
            intervals[i][0] = sc.nextInt();  // Start time
            intervals[i][1] = sc.nextInt();  // End time
        }

        // Print result
        System.out.println(minMeetingRooms(intervals));
    }
}
```

---

1. Program Title

Rotate Matrix by 90 Degrees Clockwise (In-Place Method)

2. Description

This program rotates a given n × n matrix by 90 degrees clockwise without using any extra matrix (in-place rotation). It first transposes the matrix and then reverses each row to achieve the rotation.

3. Purpose of the Program

The purpose of this program is to efficiently rotate a square matrix by 90 degrees clockwise using an optimized in-place approach with O(1) extra space.

4. Input

First input: Integer n (size of matrix)

Next n lines: n integers each (matrix elements.

5. Output

Prints the rotated matrix (90° clockwise).

6. Algorithm / Logic

Step 1: Transpose the Matrix

Swap `matrix[i][j]` with `matrix[j][i]`

Traverse only upper triangle (`j = i + 1`) to avoid double swapping.

Step 2: Reverse Each Row

For each row:

- Use two pointers (`left`, `right`)
- Swap elements until `left < right`

This results in 90 clockwise rotation

Variable		Purpose
`intervals[][]`	=	Stores meeting intervals
`starts[]`	=	Stores start times
`ends[]`	=	Stores end times
`rooms`	=	Current rooms in use
`maxRooms`	=	Maximum rooms required
`s, e`	=	Two pointers for traversal
`n`	=	Number of meetings

9. Sample Input

3

1 2 3
4 5 6
7 8 9

10. Sample Output

7 4 1
8 5 2
9 6 3

11. Time Complexity

$O(n^2)$
Because every element is visited once.

12. Space Complexity

O(1)
Rotation is done in-place without extra matrix.

13. Java Program (With Comments)

```
import java.util.*;

public class rotateMatrix {

    // Method to rotate matrix by 90 degrees clockwise
    public static void rotateMatrix(int[][] matrix, int n) {

        // Step 1: Transpose the matrix
        for(int i = 0; i < n; i++) {
            for(int j = i + 1; j < n; j++) {

                // Swap matrix[i][j] with matrix[j][i]
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        // Step 2: Reverse each row
        for(int i = 0; i < n; i++) {

            int left = 0;
            int right = n - 1;

            // Swap elements in the row
            while(left < right) {
```

```
            int temp = matrix[i][left];
            matrix[i][left] = matrix[i][right];
            matrix[i][right] = temp;

            left++;
            right--;
        }
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();   // Read size of matrix
    int[][] matrix = new int[n][n];

    // Read matrix elements
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            matrix[i][j] = sc.nextInt();
        }
    }

    // Rotate the matrix
    rotateMatrix(matrix, n);

    // Print rotated matrix
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
}
```

---

1. Program Title

Longest Consecutive Sequence Using HashSet

2. Description

This program finds the length of the longest consecutive sequence in an unsorted array of integers. The sequence elements do not need to be adjacent in the array. The program uses a `Hashset` to achieve an optimized time complexity of O(n).

3. Purpose of the Program

The purpose of this program is to efficiently determine the longest sequence of consecutive integers present in an unsorted array without sorting the array.

4. Input (I/P)

First input: Integer `n` (size of array)

Second input: `n` integers (array elements)

5. Output (O/P)

Prints the length of the longest consecutive sequence.

6. Algorithm / Logic

1. Insert all elements of the array into a `HashSet`.
2. Initialize `longest = 0`.
3. Traverse each element in the set:

   Check if `(num - 1)` is not in the set.

   If not, then `num` is the start of a sequence.

4. Count consecutive numbers using a loop:
   - While `(currentNum + 1)` exists in the set, increment count.
5. Update `longest` using `Math.max(longest, count)`.
6. Return `longest`.

7. Variables Used

Variable		Purpose
`nums[]`	=	Stores array elements
`set`	=	Stores unique elements
`longest`	=	Stores longest sequence length
`currentNum`	=	Tracks current sequence number

`count`	=	Counts length of current sequence
`n`	=	Size of array
`arr[]`	=	Input array
`sc`	=	Used for input

8. Functions / Methods Used

`longestConsecutive()` = Finds longest sequence length

`main()` = Handles input and output

`HashSet.add()` = Adds element to set

`HashSet.contains()` = Checks element existence

`Math.max()` = Finds maximum value

`Scanner` =Takes user input

9. Sample Input

6
100 4 200 1 3 2

10. Sample Output

4

11. Time Complexity

O(n)
Each element is inserted and checked in the HashSet once.

13. Java Program (With Comments)

```
import java.util.*;

public class Longest {

    // Method to find longest consecutive sequence
    public static int longestConsecutive(int[] nums) {

        // HashSet to store unique elements
        HashSet<Integer> set = new HashSet<>();
```

```
        // Add all elements into set
        for (int num : nums) {
            set.add(num);
        }

        int longest = 0;  // Stores longest sequence length

        // Traverse through set
        for (int num : set) {

            // Check if number is start of sequence
            if (!set.contains(num - 1)) {

                int currentNum = num;
                int count = 1;

                // Count consecutive numbers
                while (set.contains(currentNum + 1)) {
                    currentNum++;
                    count++;
                }

                // Update longest sequence
                longest = Math.max(longest, count);
            }
        }

        return longest;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();  // Size of array
        int[] arr = new int[n];

        // Read array elements
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Print result
        System.out.println(longestConsecutive(arr));
    }
}
```

---

1. Program Title

Top K Frequent Elements Using HashMap

2. Purpose of the Program

The purpose of this program is to efficiently determine the top `k` most frequently occurring elements in an array using optimized data structures.

4. Input (I/P)

- First input: Integer `n` (size of array)
- Second input: `n` integers (array elements)
- Third input: Integer `k` (number of most frequent elements required)

5. Output (O/P)

Prints the `k` most frequent elements (order may vary).

6. Algorithm / Logic

1. Create a `HashMap` to store frequency of each element.
2. Traverse the array:
    - Update frequency using `getOrDefault()`.
3. Create a `PriorityQueue` (Min-Heap) with comparator based on frequency.
4. Traverse through map keys:
    - Add element to heap.
    - If heap size exceeds `k`, remove the smallest frequency element.
5. Extract elements from heap into result array.
6. Print the result.